Geometric Integration with Event Detection for Simulating Coordinate-Free State-Space Rigid Body Dynamics with Impacts

This repo is an extension of another repo I created, `geometric_rigid_body_dynamics`, in which I demonstrated how a geometric integrator is used to simulate rigid body dynamics directly in terms of homogeneous transformation matrices, twists, and wrenches, without having to deal with multiple coordinate charts.

In this repo, I add two more features:

- event detection: detects when the rigid body impacts the ground;
- impact map: updates the state of the rigid body after impact.

I came across a preprint of a similar paper: "Lie Group Variational Collision Integrators for a Class of Hybrid Systems" by Tran and Leok. It takes a slightly different approach to the problem but is a helpful reference!

1 Hybrid Dynamics

We model and simulate a bouncing ball as a hybrid system, i.e., a system consisting of continuous dynamics (the differential equations of motion of the ball when in free fall) and discrete dynamics (the algebraic equations that relate the post-impact state of the ball to its pre-impact state). At a high level of abstraction, the hybrid dynamics are

$$\dot{x} = f(x), \qquad \text{if } h(x) \neq 0, \tag{1a}$$

$$x^+ = \Delta(x^-), \qquad \text{if } h(x) = 0, \tag{1b}$$

where f is a vector field that defines the continuous-time dynamics, $h(x)$ is the *event function* (a function of the state x that returns zero when impact occurs) and Δ is a function that maps the pre-impact state x^- to the post-impact state x^+.

In this repo, the event function h will return the height above ground of the lowest point on the ball, and the impact map Δ will arise from conservation of momentum. My focus in this repo is on how to incorporate event detection in a Runge-Kutta geometric integrator, rather than on the details of the event function, so I will start with a simple case: a spherical ball; as such, the distance between the ground and the bottom of the ball is not affected by the ball's orientation. Simulation of an ellipsoidal ball is likely to produce more interesting (maybe even chaotic!) results but I'll save that for later.

1.1 Continuous Dynamics

In this repo, I am using the geometric integrator proposed by Crouch and Grossman (J. Nonlinear Sci., 1993) to simulate the motion of a rigid body in 3D space. The body has 6 degrees of freedom (DOF), but in order to avoid the pitfalls associated with choosing a set of 6 generalized coordinates, I am using a coordinate-free approach by working directly with the 4x4 homogeneous transformation matrix that represents the body's configuration in space.

Denote this transformation matrix $T_{\mathrm{wb}} \in \mathbb{SE}(3)$, where $\{\mathrm{w}\}$ is the world frame and $\{\mathrm{b}\}$ is the body frame, i.e., a frame oriented along the body's principal axes of inertia and whose origin is located at the body's center of mass (COM). The special Euclidean group $\mathbb{SE}(3)$ is a *Lie group*—a kind of differentiable manifold that has special geometry that is not preserved by generic numerical integration algorithms like 4th-order Runge-Kutta, which operate on the n-dimensional Euclidean space $\mathbb{R}^n$. Thus, a different approach is required to faithfully simulate coordinate-free rigid body dynamics. To do this, Crouch and Grossman propose a modified 4th-order Runge-Kutta scheme, which I'll call CG4.

Represented as a system of first-order ordinary differential equations (ODEs), the equations of motion for a rigid body in space are

$$\frac{\mathrm{d}}{\mathrm{d}t} \underbrace{\begin{bmatrix} T_{\mathrm{wb}}(t) \\ \mathcal{V}_{\mathrm{b}}(t) \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} T_{\mathrm{wb}}(t)\left[\mathcal{V}_{\mathrm{b}}(t)\right] \\ \mathcal{K}\left(t, T_{\mathrm{wb}}(t), \mathcal{V}_{\mathrm{b}}(t)\right) \end{bmatrix}}_{f(x)}, \tag{2}$$

where

$$\mathcal{K}\left(t, T_{\mathrm{wb}}(t), \mathcal{V}_{\mathrm{b}}(t)\right) = \mathcal{G}_{\mathrm{b}}^{-1}\left(\left[\mathrm{ad}_{\mathcal{V}_{\mathrm{b}}(t)}\right]^{\top}\mathcal{G}_{\mathrm{b}}\mathcal{V}_{\mathrm{b}}(t) + \mathcal{F}_{\mathrm{b}}(t)\right). \tag{3}$$

In these equations,

- $T_{\mathrm{wb}} \in \mathbb{SE}(3)$ is the configuration of the body's COM frame {b} relative to the inertial world frame {w}.
- $[\mathcal{V}_{\mathrm{b}}] \in \mathfrak{se}(3)$ is the matrix representation of the body twist $\mathcal{V}_{\mathrm{b}} \in \mathbb{R}^6$, where $\mathcal{V}_{\mathrm{b}} = \left[\omega_{\mathrm{b}}^{\top}, v_{\mathrm{b}}^{\top}\right]^{\top}$, where $\omega_{\mathrm{b}} \in \mathbb{R}^3$ and $v_{\mathrm{b}} \in \mathbb{R}^3$ are the angular and linear velocity, respectively, represented in the COM frame {b}.
- $\mathcal{G}_{\mathrm{b}} \in \mathbb{R}^{6\times 6}$ is the body's inertia tensor; $\mathcal{G}_{\mathrm{b}}$ is a diagonal matrix with $(\mathcal{I}_{xx}, \mathcal{I}_{yy}, \mathcal{I}_{zz}, \mathfrak{m}, \mathfrak{m}, \mathfrak{m})$ along the diagonal, where $\mathcal{I}_{xx}$, $\mathcal{I}_{yy}$, and $\mathcal{I}_{zz}$ are the inertias about the principal axes and $\mathfrak{m}$ is the body's mass.
- $\mathcal{F}_{\mathrm{b}} \in \mathbb{R}^6$ is the net external wrench on the body, expressed in the COM frame. $\mathcal{F}_{\mathrm{b}} = \left[m_{\mathrm{b}}^{\top}, f_{\mathrm{b}}^{\top}\right]^{\top}$, where $m_{\mathrm{b}} \in \mathbb{R}^3$ and $f_{\mathrm{b}} \in \mathbb{R}^3$ are the moment and force, respectively, in the COM frame.
- $\left[\mathrm{ad}_{\mathcal{V}_{\mathrm{b}}(t)}\right]$ is a 6x6 matrix used to express the rate of change of spatial momentum $\mathcal{P}_{\mathrm{b}} = \mathcal{G}_{\mathrm{b}}\mathcal{V}_{\mathrm{b}}$ due to the twist $\mathcal{V}_{\mathrm{b}}$.

Note that the state space of the first-order ordinary differential equation (ODE) above is $\mathcal{X} = \mathbb{SE}(3) \times \mathbb{R}^6$, and the state $x \in \mathcal{X}$ is the pair $(T_{\mathrm{wb}}, \mathcal{V}_{\mathrm{b}})$.

1.2 Discrete Dynamics

1.2.1 Event Function and Event Detection

As stated above, our event function $h : \mathbb{SE}(3) \times \mathbb{R}^6 \to \mathbb{R}$ is the distance between the bottom of the ball and the ground. If, while integrating Equation (2) from t_k to t_{k+1}, we observe that $h(x(t_{k+1})) \neq h(x(t_k))$, we must find the event time $t_{\mathrm{e}} \in [t_k, t_{k+1}]$ such that $h(x(t_{\mathrm{e}}) = 0)$.

We consider two approaches to event finding:

- Bisection root finding;
- Newton-Raphson root finding.

While the bisection method may take more iterations to converge to an acceptable estimate of t_{e}, it is guaranteed to return an estimate in the time window $[t_k, t_{k+1}]$. Conversely, while the Newton-Raphson method may be more elegant, it is possible it will return an estimate outside $[t_k, t_{k+1}]$, if $\mathrm{d}h/\mathrm{d}t$ is sufficiently small and $\mathrm{d}^2h/\mathrm{d}t^2$ is sufficiently large.

Bisection Root Finding Searching for roots of the event function via bisection is simple:

1. Given a time window $[t_k, t_{k+1}]$ for which $\mathrm{sign}(h(t_k)) \neq \mathrm{sign}(h(t_{k+1}))$, divide the time window in half to produce two new time windows: $[t_k, t_k + \Delta t/2]$ and $[t_k + \Delta t/2, t_{k+1}]$, where $\Delta t = t_{k+1} - t_k$.
2. If $\mathrm{sign}(h(t_k)) \neq \mathrm{sign}(h(t_k + \Delta t/2))$, repeat step 1 for the new time window $[t_k, t_k + \Delta t/2]$. Otherwise, repeat step 1 for the new time window $[t_k + \Delta t/2, t_{k+1}]$.
3. Iterate until some convergence criterion is satisfied, or until the maximum number of iterations is reached.

Newton-Raphson Root Finding Given an initial guess at the event time $t_0 \in [t_k, t_{k+1}]$, the Taylor series expansion of $h(x(t))$ at t_0 is

$$h(x(t)) = h(x(t_0)) + \mathcal{L}_f h(x(t_0))\,(t - t_n) + \text{h.o.t}, \tag{4}$$

where $\mathcal{L}_f h(x(t_0))$ is the Lie derivative of h along f evaluated at the state x at time t_0:

$$\begin{aligned}\mathcal{L}_f h(x(t_0)) &= \left.\frac{\mathrm{d}h}{\mathrm{d}t}\right|_{t_0} \\ &= \left.\frac{\partial h}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t}\right|_{t_0} \\ &= \frac{\partial h}{\partial x} f(x(t_0)),\end{aligned} \tag{5}$$

where f is the right-hand side of Equation (2). Neglecting higher-order terms, setting $h(x(t)) = 0$, and solving for t, we obtain a first-order approximation for the event time:

$$t \approx t_0 - \left(\mathcal{L}_f h(x(t_0))\right)^{-1} h(x(t_0)). \tag{6}$$

Updating our initial guess with this new t and repeating the above, we get the following iteration:

$$t_{n+1} = t_n - \left(\mathcal{L}_f h(x(t_n))\right)^{-1} h(x(t_n)), \tag{7}$$

which we repeat until some convergence criterion is satisfied or until the number of iterations reaches a user-specified maximum.

Simple Case - Sphere: The position of the bottom of a sphere with radius r is

$$h(T_{\mathrm{wb}}, \mathcal{V}_{\mathrm{b}}) = a^\top T_{\mathrm{wb}} b - r, \tag{8}$$

where $a = [0, 0, 1, 0]^\top$ and $b = [0, 0, 0, 1]^\top$. Intuitively, $\mathrm{d}h/\mathrm{d}t$ should be the velocity of the center of mass (COM) in the $\hat{z}_{\mathrm{w}}$ direction: $\mathrm{d}h/\mathrm{d}t = a^\top \dot{T}_{\mathrm{wb}} b$. For this simple case,

$$\frac{\partial h}{\partial T_{\mathrm{wb}}} = ab^\top = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \text{ and } \frac{\partial h}{\partial \mathcal{V}_{\mathrm{b}}} = 0_{1\times 6}. \tag{9}$$

It is worth noting that the 4-by-4 matrix above is G_3, one of the six generators of the Lie algebra $\mathfrak{se}(3)$:

$$G_1 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad G_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad G_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

$$G_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad G_5 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad G_6 = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

This does indeed pull out just the $\hat{z}_{\mathrm{w}}$-component of $\dot{T}_{\mathrm{wb}}$, i.e., the COM velocity in the vertical direction. It helps to think about the following pair of operators for "unpacking" a matrix into a vector and for "packing" a vector into a matrix:

$$\begin{aligned}\mathrm{vec} &: \mathbb{R}^{n\times m} \to \mathbb{R}^{nm}, \\ \mathrm{unvec} &: \mathbb{R}^{nm} \to \mathbb{R}^{n\times m}.\end{aligned}$$

I will follow the row-major convention throughout this document, i.e.,

$$\mathrm{vec}\left(\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}\right) = \begin{bmatrix} a_{11} & a_{12} & a_{21} & a_{22} \end{bmatrix}^\top.$$

For our simple event function, $\mathrm{vec}\,(\partial h/\partial x) = [0_{1\times 11}, 1, 0_{1\times 10}]$.

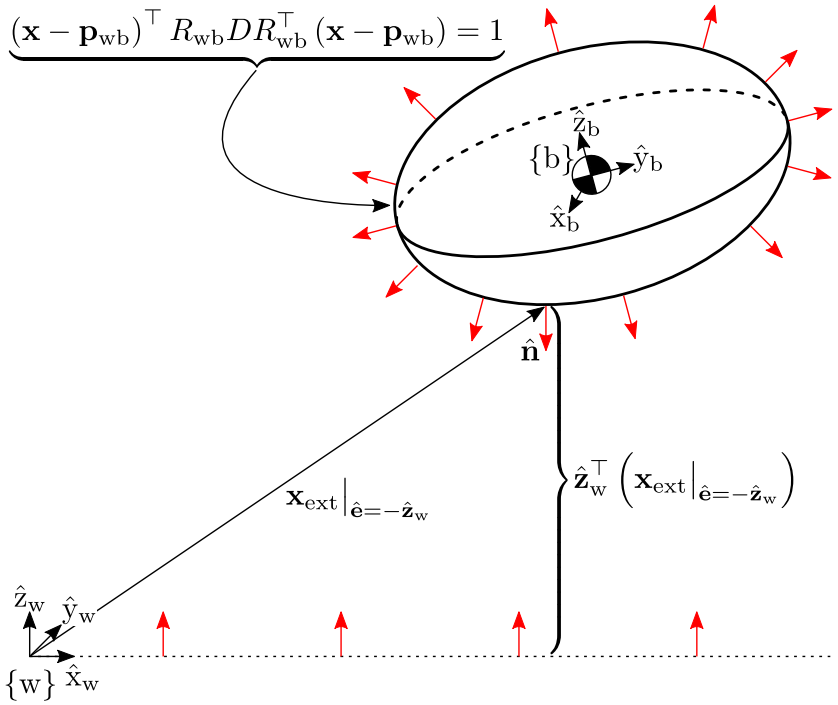


Figure 1: Ellipsoid kinematics. The configuration of the ellipsoid is given by T_{wb}. The lowest point on the ellipse, located at $\mathbf{x}_{\mathrm{ext}}\big|_{\hat{\mathbf{e}}=-\hat{\mathbf{z}}_{\mathrm{w}}}$ in the $\{\mathrm{w}\}$-frame, has unit surface normal $\hat{\mathbf{n}}$, which points directly down. The distance from this point to the ground is given by the dot product of $\hat{\mathbf{z}}$ and $\mathbf{x}_{\mathrm{ext}}\big|_{\hat{\mathbf{e}}=-\hat{\mathbf{z}}_{\mathrm{w}}}$.

More Complex Case: Ellipsoid Suppose instead we want to know the distance between the lowest point on an ellipsoid and the ground, as shown in Figure 1. The ellipsoid is defined implicitly by the equation

$$(\mathbf{x}-\mathbf{p}_{\mathrm{wb}})^{\top} R_{\mathrm{wb}} D R_{\mathrm{wb}}^{\top} (\mathbf{x}-\mathbf{p}_{\mathrm{wb}}) = 1, \tag{10}$$

where $\mathbf{x} \in \mathbb{R}^3$, $(R_{\mathrm{wb}}, \mathbf{p}_{\mathrm{wb}}) \in \mathbb{SE}(3)$, and $D \in \mathbb{R}^{3\times 3}$ is a diagonal matrix whose entries are the reciprocals of the squares of the semi-axes. For convenience, we will write (10) in the following more concise form:

$$\mathbf{y}^{\top} A \mathbf{y} = 1, \tag{11}$$

where $\mathbf{y} = \mathbf{x} - \mathbf{p}_{\mathrm{wb}}$ and $A = R_{\mathrm{wb}} D R_{\mathrm{wb}}^{\top}$. Note that A is symmetric. We will formulate the event function in a more general setting—how close is the ellipsoid to a (flat) surface whose unit normal vector is $\hat{\mathbf{e}}$?—and we will then substitute the unit normal vector corresponding to level ground, $\hat{\mathbf{e}} = -\hat{\mathbf{z}}_{\mathrm{w}} = [0, 0, -1]^{\top}$.

For an ellipsoid, the unit vector normal to the surface at the point $\mathbf{y}$ furthest in the $\hat{\mathbf{e}}$-direction is simply $\hat{\mathbf{e}}$. The matrix A in Equation (11) stretches the vector $\mathbf{y}$ by some amount μ and rotates it to align with $\hat{\mathbf{e}}$:

$$A\mathbf{y} = \mu\hat{\mathbf{e}}. \tag{12}$$

Solving for $\mathbf{y}$, we have

$$\mathbf{y} = \mu A^{-1}\hat{\mathbf{e}}. \tag{13}$$

Substituting this result into Equation (11), we have

$$\begin{aligned} 1 &= \mathbf{y}^{\top} A \mathbf{y} \\ &= \left(\mu A^{-1}\hat{\mathbf{e}}\right)^{\top} A \left(\mu A^{-1}\hat{\mathbf{e}}\right) \\ &= \mu^2 \hat{\mathbf{e}}^{\top} A^{-\top} A A^{-1} \hat{\mathbf{e}} \\ &= \mu^2 \hat{\mathbf{e}}^{\top} A^{-1} \hat{\mathbf{e}}. \end{aligned}$$

The simplification leading to the last line is possible because A is symmetric. Solving for the scaling factor μ, we get

$$\mu = \frac{1}{\sqrt{\hat{\mathbf{e}} A^{-1} \hat{\mathbf{e}}}}. \tag{14}$$

Substituting this expression for μ into Equation (13), we get

$$\mathbf{y} = \frac{A^{-1}\hat{\mathbf{e}}}{\sqrt{\hat{\mathbf{e}}A^{-1}\hat{\mathbf{e}}}}. \tag{15}$$

We substitute $\mathbf{x}_{\text{ext}}\big|_{\hat{\mathbf{e}}} - \mathbf{p}_{\text{wb}}$ for $\mathbf{y}$ and $R_{\text{wb}}DR_{\text{wb}}^{\top}$ for A and solve for $\mathbf{x}_{\text{ext}}\big|_{\hat{\mathbf{e}}}$, the world-frame position of the point on the ellipsoid that is farthest from the origin of {w} in the $\hat{\mathbf{e}}$-direction:

$$\mathbf{x}_{\text{ext}}\big|_{\hat{\mathbf{e}}} = \mathbf{p}_{\text{wb}} + \frac{R_{\text{wb}}D^{-1}R_{\text{wb}}^{\top}\hat{\mathbf{e}}}{\sqrt{\hat{\mathbf{e}}^{\top}R_{\text{wb}}D^{-1}R_{\text{wb}}^{\top}\hat{\mathbf{e}}}}. \tag{16}$$

Finally, to use Equation (16) to find the point on the ellipsoid closest to the ground, we substitute $\hat{\mathbf{e}} = -\hat{\mathbf{z}}_{\text{w}}$ into Equation (16). The event function for the ellipsoid impacting ground is then

$$h\left(T_{\text{wb}}, \mathcal{V}_{\text{b}}\right) = \hat{\mathbf{z}}_{\text{w}}^{\top}\left(\mathbf{x}_{\text{ext}}\big|_{\hat{\mathbf{e}}=-\hat{\mathbf{z}}_{\text{w}}}\right) = 0. \tag{17}$$

1.2.2 Impact Map